

SOURCE CODE DEPOSIT

COPYRIGHT REGISTRATION APPLICATION

Software Title:	HomeFlow
Nature of Work:	Mobile Application Software (Flutter / Dart)
Author / Owner:	[YOUR FULL LEGAL NAME / COMPANY NAME]
Date of Creation:	2025 - 2026
Platform:	Android and iOS (cross-platform)

The following pages contain representative source code excerpts from the HomeFlow mobile application. This deposit is submitted in support of a copyright registration application. All code was authored originally by the applicant and has not been published or licensed to any third party.

TABLE OF CONTENTS

1. Application Entry Point / lib/main.dart

2. Data Model - Supply Item / lib/models/supply_item.dart

3. Data Model - Shopping Request / lib/models/shopping_request.dart

4. State Management - Supply Provider / lib/providers/supply_provider.dart

5. Business Logic - Retailer Price Comparison / lib/services/retailer_catalog_service.dart

6. Authentication Service / lib/services/supabase_auth_service.dart

7. Backend Sync Service / lib/services/sync_service.dart

Declaration and Signature

1. Application Entry Point

File: lib/main.dart

Application bootstrap, dependency injection, and client-side router configuration.

```
import 'package:flutter/material.dart';
import 'package:go_router/go_router.dart';
import 'package:provider/provider.dart';
import 'providers/supply_provider.dart';
import 'providers/price_compare_provider.dart';
import 'services/supabase_service.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await SupabaseService.initialize();
  runApp(const HomeFlowApp());
}

class HomeFlowApp extends StatelessWidget {
  const HomeFlowApp({super.key});

  static final GoRouter _router = GoRouter(
    initialLocation: '/admin/dashboard',
    routes: [
      GoRoute(
        path: '/admin/dashboard',
        builder: (context, state) => const AdminPanelScreen(
          selectedIndex: 0,
          child: AdminDashboardPage(),
        ),
      ),
      GoRoute(
        path: '/admin/households/:id',
        builder: (context, state) => AdminPanelScreen(
          selectedIndex: 1,
          child: AdminHouseholdDetailPage(
            householdId: state.pathParameters['id'] ?? '',
          ),
        ),
      ),
    ],
  );
}
```

2. Data Model - Supply Item

File: lib/models/supply_item.dart

Core domain model for a tracked household supply item with usage logs and role visibility.

```
enum SupplyStatus { enough, runningLow, veryLow, finished }

class SupplyUsageEntry {
  final DateTime date;
  final double quantity;
  final String? notes;
  final double? price;
  final String? loggedByName;

  Map<String, dynamic> toJson() => {
    'date': date.toIso8601String(),
    'quantity': quantity,
    if (notes != null) 'notes': notes,
    if (price != null) 'price': price,
    if (loggedByName != null) 'loggedByName': loggedByName,
  };
}

class SupplyItem {
  final String id;
  final String householdId;
  final String name;
  final String category;
  final String unitType;
  final SupplyStatus status;
  final String? preferredBrand;
  final bool isGas;
  // When true, hidden from house managers - owner eyes only
  final bool isOwnerOnly;
  final List<SupplyUsageEntry> usageLogs;
  final DateTime? statusUpdatedAt;
  final String? statusUpdatedByName;
}
```

3. Data Model - Shopping Request

File: lib/models/shopping_request.dart

Models the complete shopping workflow including urgency, approval states, and purchase types.

```
enum ShoppingUrgency { neededSoon, neededToday, critical }
enum ShoppingStatus { requested, seen, approved, purchased, deferred }

// managerRequest - manager noticed low stock, awaits owner approval
// ownerPurchase - owner personal buy list, no approval needed
// managerDirectBuy - manager bought immediately (emergency)
enum PurchaseType { managerRequest, ownerPurchase, managerDirectBuy }

class ShoppingRequest {
  final String      id;
  final String      householdId;
  final String      itemName;
  final String      quantity;
  final String      category;
  final ShoppingUrgency urgency;
  final ShoppingStatus status;
  final PurchaseType purchaseType;
  final bool        autoApproved;
  final String?     buyAnywayReason;
  final String      requestedByUserId;
  final String      requestedByName;
  final DateTime     requestedAt;

  bool get needsApproval =>
    purchaseType == PurchaseType.managerRequest &&
    (status == ShoppingStatus.requested ||
     status == ShoppingStatus.seen);

  bool get isOwnerPurchase => purchaseType == PurchaseType.ownerPurchase;
  bool get wasBuyAnyway    => buyAnywayReason != null;
}
```

4. State Management - Supply Provider

File: lib/providers/supply_provider.dart

ChangeNotifier managing supply tracking, shopping requests, Supabase sync and local cache.

```
class SupplyProvider extends ChangeNotifier {
  List<SupplyItem>      _supplies      = [];
  List<ShoppingRequest> _shoppingRequests = [];

  List<SupplyItem> visibleSupplies({required bool isOwner}) =>
    isOwner ? _supplies
      : _supplies.where((s) => !s.isOwnerOnly).toList();

  List<SupplyItem> get finishedSupplies =>
    _supplies.where((s) => s.status == SupplyStatus.finished).toList();

  List<ShoppingRequest> get pendingRequests => _shoppingRequests
    .where((r) => r.purchaseType == PurchaseType.managerRequest &&
      (r.status == ShoppingStatus.requested ||
        r.status == ShoppingStatus.seen))
    .toList();

  List<ShoppingRequest> get approvedRequests => _shoppingRequests
    .where((r) => r.purchaseType == PurchaseType.managerRequest &&
      r.status == ShoppingStatus.approved)
    .toList();

  Future<void> loadData(String householdId) async {
    final remoteSupplies = await SyncService.fetchAll(
      'app_supplies', householdId, SupplyItem.fromJson);
    if (remoteSupplies != null) {
      _supplies = remoteSupplies;
      final prefs = await SharedPreferences.getInstance();
      await prefs.setString('supplies_$householdId',
        jsonEncode(_supplies.map((s) => s.toJson()).toList()));
    }
    notifyListeners();
  }

  Future<void> deleteShoppingRequest(
    String requestId, String householdId) async {
    _shoppingRequests.removeWhere((r) => r.id == requestId);
    notifyListeners();
    await SyncService.save('app_shopping_requests', householdId,
      _shoppingRequests, (r) => r.toJson());
  }

  Future<void> removeSupplyItem(String supplyId, String householdId) async {
    _supplies.removeWhere((s) => s.id == supplyId);
    notifyListeners();
    await SyncService.save('app_supplies', householdId,
      _supplies, (s) => s.toJson());
  }
}
```

5. Business Logic - Retailer Price Comparison

File: lib/services/retailer_catalog_service.dart

Original 4-pass fuzzy matching algorithm resolving typed item names to Carrefour/Naivas catalog entries.

```
// Pass 1: Exact phrase containment          (score 3.0)
// Pass 2: Compound-word no-space match     (score 2.5)
//           e.g. "cornflakes" matches "corn flakes"
// Pass 3: Token-level substring            (score 1.5)
// Pass 4: Per-token Levenshtein distance   (score 1.2)

static List<RetailerQuote> quotesForItem(
  String itemName, {String? preferredBrand}) {

  final normalized = itemName.toLowerCase().trim();
  final compact    = normalized.replaceAll(RegExp(r'\s+'), '');
  final tokens     = normalized.split(RegExp(r'[/\s/]+')).
    where((t) => t.length > 2).toList();
  final best = <RetailerCode, ({_CatalogEntry entry, double score})>{};

  for (final entry in _catalog) {
    double score = 0;
    for (final kw in entry.keywords) {
      if (normalized.contains(kw) || kw.contains(normalized)) {
        score = score < 3.0 ? 3.0 : score; continue;
      }
      final kwc = kw.replaceAll(' ', '');
      if (kwc.length > 3 &&
        (compact.contains(kwc) || kwc.contains(compact))) {
        score = score < 2.5 ? 2.5 : score; continue;
      }
      for (final t in tokens)
        if (t.length > 2 && (kw.contains(t) || t.contains(kw)))
          score = score < 1.5 ? 1.5 : score;
      for (final t in tokens) {
        if (t.length < 4) continue;
        for (final w in kw.split(' ')) {
          if (w.length < 4) continue;
          final maxD = (t.length >= 7 || w.length >= 7) ? 2 : 1;
          if (_editDistance(t, w) <= maxD)
            score = score < 1.2 ? 1.2 : score;
        }
      }
    }
    if (score == 0) continue;
    final ex = best[entry.retailer];
    if (ex == null || score > ex.score)
      best[entry.retailer] = (entry: entry, score: score);
  }
  return RetailerCode.values.map((code) {
    final f = best[code];
    if (f == null) return RetailerQuote(retailerCode: code,
      matchType: MatchType.notFound, isAvailable: false, confidenceScore: 0);
    return RetailerQuote(retailerCode: code, productName: f.entry.productName,
      price: f.entry.price, currency: 'KES', isAvailable: true,
      matchType: f.entry.matchType, confidenceScore: f.entry.confidence);
  }).toList();
}
```

```
}

// Wagner-Fischer Levenshtein
static int _editDistance(String a, String b) {
    if ((a.length - b.length).abs() > 3) return 99;
    final dp = List.generate(
        a.length+1, (i) => List.generate(b.length+1, (j) => i==0 ? j : j==0 ? i : 0));
    for (var i=1; i<=a.length; i++)
        for (var j=1; j<=b.length; j++)
            dp[i][j] = a[i-1]==b[j-1] ? dp[i-1][j-1]
                : 1+[dp[i-1][j], dp[i][j-1], dp[i-1][j-1]].reduce((x,y)=>x<y?x:y);
    return dp[a.length][b.length];
}
```

6. Authentication Service

File: lib/services/supabase_auth_service.dart

Wraps Supabase GoTrue auth: sign-up, sign-in, OTP verification, role inference from metadata.

```
class SupabaseAuthService {
  bool get hasActiveSession =>
    SupabaseService.auth.currentUser != null;

  Future<AuthResponse> signInWithPassword({
    required String email,
    required String password,
  }) => SupabaseService.auth.signInWithPassword(
    email: email, password: password);

  Future<AuthResponse> signUp({
    required String email,
    required String password,
    Map<String, dynamic>? data,
  }) => SupabaseService.auth.signUp(
    email: email, password: password, data: data);

  Future<void> signOut() async {
    try { await SupabaseService.auth.signOut(); }
    catch (e) { debugPrint('signOut skipped: $e'); }
  }

  // Infer household role from Supabase user metadata
  UserRole inferRoleFromMetadata(User user) {
    final role = user.userMetadata?['role']?.toString();
    return role == UserRole.houseManager.name
      ? UserRole.houseManager : UserRole.owner;
  }

  Future<AuthResponse> verifyOtp({
    required String email, required String token,
  }) => SupabaseService.auth.verifyOTP(
    email: email, token: token, type: OtpType.signup);

  Future<void> resendOtp({required String email}) =>
    SupabaseService.auth.resend(type: OtpType.signup, email: email);
}
```


7. Backend Sync Service

File: lib/services/sync_service.dart

Two-way sync between local SharedPreferences cache and Supabase with offline fallback.

```
class SyncService {
  static final _db = SupabaseService.client;
  static bool get isAuthenticated => _db.auth.currentUser != null;

  // Fetch all rows for [table] / [householdId].
  // Returns null so callers can fall back to local cache.
  static Future<List<T>?> fetchAll<T>(
    String table, String householdId,
    T Function(Map<String, dynamic>) fromJson,
  ) async {
    if (!isAuthenticated) return null;
    try {
      final rows = await _db.from(table).select()
        .eq('household_id', householdId).order('created_at');
      return rows.map((r) => fromJson(r as Map<String, dynamic>)).toList();
    } catch (e) {
      debugPrint('[SyncService] fetchAll $table: $e'); return null;
    }
  }

  static Future<void> save<T>(
    String table, String householdId, List<T> items,
    Map<String, dynamic> Function(T) toJson,
  ) async {
    if (!isAuthenticated) return;
    try {
      await _db.from(table).upsert(items.map((i) =>
        {...toJson(i), 'household_id': householdId}).toList());
    } catch (e) { debugPrint('[SyncService] save $table: $e'); }
  }

  static Future<void> ensureHouseholdMember(
    String householdId, {String? fullName, String? displayEmail}) async {
    if (!isAuthenticated) return;
    await _db.from('app_household_members').upsert({
      'user_id': _db.auth.currentUser!.id,
      'household_id': householdId,
      if (fullName != null) 'full_name': fullName,
      if (displayEmail != null) 'display_email': displayEmail,
    });
  }
}
```

DECLARATION

I declare that:

1. The source code excerpts reproduced in this document are original works authored by me/us.

2. The code

Applicant Name: _____

Signature: _____

Date: 26 April 2026

ID / Reg. No.: _____

- End of Source Code Deposit Document -